

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: N. Samhavya

Roll No.: A24126552099

Date: 19-09-2026

TASK 1 — INTERACTIVE PRODUCT CALCULATOR (CO-1)

1. TITLE

Develop an Interactive Product Price Calculator Webpage Using HTML, CSS, JavaScript, and DOM Event Listeners

2. AIM

To design and implement an interactive webpage containing product name, unit price, and quantity inputs that dynamically calculates and renders subtotal, tax (18% GST), and total order price in real time upon any input modification without refreshing the webpage.

3. OBJECTIVE

The objectives of this task are:

- To create a structured, accessible HTML5 user interface using semantic form controls.
- To capture real-time user input events using JavaScript `addEventListener` (input and change events).
- To perform arithmetic operations including subtotal calculation, percentage tax computation, and grand total evaluation.
- To dynamically manipulate DOM tree elements using `textContent` to reflect calculated financial values instantaneously without refreshing the page.
- To implement input validation ensuring only positive numerical values are processed and display user-friendly error alerts for invalid states.
- To provide quick preset options and a form reset mechanism.

4. THEORY

Dynamic web applications rely on asynchronous client-side scripting to manipulate the Document Object Model (DOM) without triggering traditional full-page HTTP reloads.

Key Concepts Used:

1. **The Document Object Model (DOM):** The DOM is an object-oriented representation of the webpage where HTML elements are modeled as nodes with configurable properties, methods, and event handlers.
2. **DOM Selection Methods:**
 - `document.getElementById(id)`: Returns an Element object matching the specified unique ID.
 - `document.querySelectorAll(selector)`: Returns a static NodeList of matching elements.
3. **Event-Driven Architecture & Event Listeners:**
 - The modern W3C standard binds events using `element.addEventListener('eventType', handlerFunction)`.
 - **input event:** Fires synchronously whenever the value of an `<input>`, `<select>`, or `<textarea>` element is altered. It offers instantaneous real-time responsiveness on keystrokes, dragging spinners, or pasting text.
 - **change event:** Fires when an alteration is committed by the user (typically upon losing focus or selecting from a dropdown).
4. **Data Type Conversion in JavaScript:**
 - Values retrieved from HTML form inputs are returned as strings (string primitive), even when `<input type="number">` is utilized.
 - `parseFloat(string)`: Parses the textual argument and returns a floating-point number.
 - `parseInt(string, radix)`: Parses a string argument and returns an integer of the specified radix (base 10).
 - `isNaN(value)`: Determines whether a value is an illegal number (Not-a-Number).
5. **DOM Updating Without Refreshing:**
 - Updating `element.textContent` alters the text node of an element immediately within the browser layout engine. This does not trigger a page refresh, preserving application state and providing seamless responsiveness.
6. **Currency Localization:**
 - `Number.prototype.toLocaleString('en-IN', options)` formats numbers with currency symbols, decimal places, and regional numbering systems (e.g., Indian Lakhs/Crores comma formatting).

5. ALGORITHM / PROCEDURE

1. Create a root HTML5 document (`index.html`) declaring standard meta tags and stylesheet links.
2. Structure a main container card featuring an input form with fields for Product Name, Unit Price (₹), and Quantity.
3. Add placeholder `<span>` tags below each input to render validation error notices.
4. Construct an Invoice Summary display card with dedicated element IDs for `displayProductName`, `displayUnitPrice`, `displayQuantity`, `displaySubtotal`, `displayTax`, and `displayGrandTotal`.
5. Add action buttons for Reset Form and preset quick-load buttons.
6. Style the layout using CSS (`style.css`) with CSS Flexbox/Grid, distinct card boundaries, clean typography, responsive styling, and animated status pulses.

7. Create a JavaScript script (script.js) and bind a DOMContentLoaded event listener to ensure safe DOM element querying.
8. Implement the calculateTotal() function:
 - Extract string values from input fields.
 - Validate that the product name is non-empty.
 - Validate that price is numeric and strictly greater than or equal to zero.
 - Validate that quantity is an integer strictly greater than or equal to 1.
 - If invalid, apply the .invalid CSS class, render an error message, and zero the total.
 - If valid,
 - calculate subtotal=unitPrice×quantity, subtotal=unitPrice×quantity, tax=subtotal×0.18, tax=subtotal×0.18, and grandTotal=subtotal+tax, grandTotal=subtotal+tax.
 - Format computed values into Indian Rupee (₹) notation and update summary element text nodes.
9. Attach input and change event listeners to the unitPrice and quantity fields.
10. Attach click handlers to preset buttons and the reset button.
11. Load the page in a web browser and verify dynamic recalculation without screen flickering or page reload.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Product Calculator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <header class="header">
      <h1>Interactive Product Calculator</h1>
      <p class="subtitle">Full Stack Web Development (23CM4121) — CO-1 Task
Demonstration</p>
    </header>
    <main class="calculator-layout">
      <!-- Product Input Form -->
      <section class="card form-card">
        <h2>Product Details</h2>
        <form id="productForm" onsubmit="return false;">
          <div class="form-group">
            <label for="productName">Product Name</label>
            <input type="text" id="productName" placeholder="e.g. Ergonomic
Keyboard" value="Wireless Gaming Mouse">
            <span class="error-msg" id="nameError"></span>
          </div>
          <div class="form-group">
```

```

        <label for="productPrice">Unit Price (₹)</label>
        <input type="number" id="productPrice" placeholder="0.00" min="0"
step="0.01" value="1499.00">
        <span class="error-msg" id="priceError"></span>
    </div>
    <div class="form-group">
        <label for="productQuantity">Quantity</label>
        <input type="number" id="productQuantity" placeholder="1" min="1"
step="1" value="2">
        <span class="error-msg" id="qtyError"></span>
    </div>
    <div class="preset-section">
        <label class="preset-label">Quick Presets:</label>
        <div class="preset-buttons">
            <button type="button" class="preset-btn" data-name="Mechanical
Keyboard" data-price="3499.00" data-qty="1">Keyboard</button>
            <button type="button" class="preset-btn" data-name="Wireless Mouse"
data-price="1499.00" data-qty="2">Mouse</button>
            <button type="button" class="preset-btn" data-name="USB-C Hub" data-
price="2750.00" data-qty="3">USB Hub</button>
        </div>
    </div>
    <div class="form-actions">
        <button type="button" id="resetBtn" class="btn btn-secondary">Reset
Form</button>
    </div>
</form>
</section>
<!-- Dynamic Invoice Summary Card -->
<section class="card summary-card">
    <h2>Live Invoice Summary</h2>
    <div class="invoice-details">
        <div class="summary-row">
            <span class="label">Product Name:</span>
            <span class="value" id="summaryName">Wireless Gaming Mouse</span>
        </div>
        <div class="summary-row">
            <span class="label">Unit Price:</span>
            <span class="value" id="summaryPrice">₹1,499.00</span>
        </div>
        <div class="summary-row">
            <span class="label">Quantity:</span>
            <span class="value" id="summaryQuantity">2</span>
        </div>
        <div class="summary-row">
            <span class="label">Subtotal:</span>
            <span class="value" id="summarySubtotal">₹2,998.00</span>
        </div>
        <div class="summary-row">
            <span class="label">GST (18%):</span>

```

```

        <span class="value" id="summaryTax">₹539.64</span>
      </div>
      <hr class="divider">
      <div class="summary-row total-row">
        <span class="label">Grand Total:</span>
        <span class="value total-amount" id="summaryTotal">₹3,537.64</span>
      </div>
    </div>
    <div class="live-indicator">
      <span class="pulse-dot"></span>
      <span id="statusText">Dynamically Calculated (No Page Reload)</span>
    </div>
  </section>
</main>
</div>
<script src="script.js"></script>
</body>
</html>

```

style.css

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
body {
  background: #f0f4f8;
  color: #2d3748;
  padding: 30px 20px;
  display: flex;
  justify-content: center;
}
.container {
  max-width: 920px;
  width: 100%;
}
.header {
  text-align: center;
  margin-bottom: 28px;
}
.header h1 {
  color: #1a365d;
  font-size: 28px;
  margin-bottom: 6px;
}
.subtitle {
  color: #718096;

```

```
    font-size: 14px;
}
.calculator-layout {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 24px;
}
@media (max-width: 768px) {
  .calculator-layout {
    grid-template-columns: 1fr;
  }
}
.card {
  background: #ffffff;
  border-radius: 12px;
  padding: 24px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
  border: 1px solid #e2e8f0;
}
.card h2 {
  font-size: 20px;
  color: #2b6cb0;
  margin-bottom: 20px;
  border-bottom: 2px solid #ebf8ff;
  padding-bottom: 8px;
}
.form-group {
  margin-bottom: 16px;
}
.form-group label {
  display: block;
  font-size: 14px;
  font-weight: 600;
  margin-bottom: 6px;
  color: #4a5568;
}
.form-group input {
  width: 100%;
  padding: 10px 14px;
  border: 1.5px solid #cbd5e0;
  border-radius: 6px;
  font-size: 15px;
  outline: none;
  transition: border-color 0.2s, box-shadow 0.2s;
}
.form-group input:focus {
  border-color: #3182ce;
  box-shadow: 0 0 0 3px rgba(49, 130, 206, 0.15);
}
.form-group input.invalid {
```

```
border-color: #e53e3e;
background-color: #fff5f5;
}
.error-msg {
  display: block;
  color: #e53e3e;
  font-size: 12px;
  margin-top: 4px;
  min-height: 16px;
}
.preset-section {
  margin: 16px 0;
}
.preset-label {
  font-size: 13px;
  font-weight: 600;
  color: #718096;
  display: block;
  margin-bottom: 6px;
}
.preset-buttons {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
}
.preset-btn {
  background: #edf2f7;
  border: 1px solid #cbd5e0;
  color: #4a5568;
  padding: 6px 12px;
  border-radius: 6px;
  font-size: 13px;
  cursor: pointer;
  transition: all 0.2s;
}
.preset-btn:hover {
  background: #e2e8f0;
  color: #2d3748;
}
.btn {
  padding: 10px 18px;
  border-radius: 6px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  border: none;
  transition: all 0.2s;
}
.btn-secondary {
  background: #edf2f7;
```

```
    color: #4a5568;
    border: 1px solid #cbd5e0;
    width: 100%;
}
.btn-secondary:hover {
    background: #e2e8f0;
}
.invoice-details {
    margin-bottom: 20px;
}
.summary-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 10px 0;
    font-size: 15px;
}
.summary-row .label {
    color: #718096;
}
.summary-row .value {
    font-weight: 600;
    color: #2d3748;
}
.divider {
    border: 0;
    border-top: 1px solid #e2e8f0;
    margin: 10px 0;
}
.total-row {
    font-size: 18px;
    padding-top: 12px;
}
.total-amount {
    color: #2b6cb0;
    font-size: 22px;
}
.live-indicator {
    display: flex;
    align-items: center;
    background: #ebf8ff;
    padding: 10px 14px;
    border-radius: 8px;
    color: #2b6cb0;
    font-size: 13px;
    font-weight: 500;
}
.pulse-dot {
    width: 10px;
    height: 10px;
```

```

background: #38a169;
border-radius: 50%;
margin-right: 10px;
animation: pulse 1.8s infinite;
}
@keyframes pulse {
  0% { transform: scale(0.95); box-shadow: 0 0 0 rgba(56, 161, 105, 0.7); }
  70% { transform: scale(1); box-shadow: 0 0 8px rgba(56, 161, 105, 0); }
  100% { transform: scale(0.95); box-shadow: 0 0 0 rgba(56, 161, 105, 0); }
}

```

script.js

```

/** Interactive Product Calculator
 * Demonstrates DOM Manipulation, Event Listeners, and Dynamic Price Calculation */
document.addEventListener('DOMContentLoaded', () => {
  // 1. DOM Element References
  const productNameInput = document.getElementById('productName');
  const unitPriceInput = document.getElementById('productPrice');
  const quantityInput = document.getElementById('productQuantity');
  const resetBtn = document.getElementById('resetBtn');
  const presetBtns = document.querySelectorAll('.preset-btn');
  // 2. Summary Card DOM References
  const summaryName = document.getElementById('summaryName');
  const summaryPrice = document.getElementById('summaryPrice');
  const summaryQuantity = document.getElementById('summaryQuantity');
  const summarySubtotal = document.getElementById('summarySubtotal');
  const summaryTax = document.getElementById('summaryTax');
  const summaryTotal = document.getElementById('summaryTotal');
  const statusText = document.getElementById('statusText');
  // 3. Error Feedback DOM References
  const nameError = document.getElementById('nameError');
  const priceError = document.getElementById('priceError');
  const qtyError = document.getElementById('qtyError');
  // 4. Helper Function: Currency Formatter
  const formatCurrency = (amount) => {
    return '₹' + amount.toLocaleString('en-IN', {
      minimumFractionDigits: 2,
      maximumFractionDigits: 2
    });
  };
  // 5. Dynamic Calculation and DOM Update Function
  function calculateTotal() {
    const name = productNameInput.value.trim();
    const price = parseFloat(unitPriceInput.value);
    const qty = parseInt(quantityInput.value, 10);
    let isValid = true;
    // Validation for Product Name
    if (!name) {

```

```

        nameError.textContent = 'Please enter a product name.';
        productNameInput.classList.add('invalid');
        isValid = false;
    } else {
        nameError.textContent = "";
        productNameInput.classList.remove('invalid');
    }
    // Validation for Unit Price
    if (isNaN(price) || price < 0) {
        priceError.textContent = 'Price must be a valid positive number.';
        unitPriceInput.classList.add('invalid');
        isValid = false;
    } else {
        priceError.textContent = "";
        unitPriceInput.classList.remove('invalid');
    }
    // Validation for Quantity
    if (isNaN(qty) || qty < 1) {
        qtyError.textContent = 'Quantity must be at least 1.';
        quantityInput.classList.add('invalid');
        isValid = false;
    } else {
        qtyError.textContent = "";
        quantityInput.classList.remove('invalid');
    }
    // DOM Update Branch
    if (isValid) {
        const subtotal = price * qty;
        const tax = subtotal * 0.18; // 18% GST calculation
        const grandTotal = subtotal + tax;
        // Dynamically update text nodes without refreshing the webpage
        summaryName.textContent = name;
        summaryPrice.textContent = formatCurrency(price);
        summaryQuantity.textContent = qty;
        summarySubtotal.textContent = formatCurrency(subtotal);
        summaryTax.textContent = formatCurrency(tax);
        summaryTotal.textContent = formatCurrency(grandTotal);
        statusText.textContent = 'Dynamically Calculated (No Page Reload)';
        statusText.style.color = '#2b6cb0';
    } else {
        // Guard state on invalid input
        summaryTotal.textContent = '₹0.00';
        statusText.textContent = 'Invalid inputs detected. Please correct errors.';
        statusText.style.color = '#e53e3e';
    }
}

// 6. Attach Event Listeners for Dynamic Real-Time Updates
productNameInput.addEventListener('input', calculateTotal);
unitPriceInput.addEventListener('input', calculateTotal);
quantityInput.addEventListener('input', calculateTotal);

```

```

// Also attach 'change' event to handle number spinner button interactions
unitPriceInput.addEventListener('change', calculateTotal);
quantityInput.addEventListener('change', calculateTotal);
// 7. Preset Selection Handler
presetBtns.forEach(btn => {
  btn.addEventListener('click', () => {
    productNameInput.value = btn.dataset.name;
    unitPriceInput.value = btn.dataset.price;
    quantityInput.value = btn.dataset.qty;
    calculateTotal();
  });
});
// 8. Form Reset Handler
resetBtn.addEventListener('click', () => {
  productNameInput.value = 'Wireless Gaming Mouse';
  unitPriceInput.value = '1499.00';
  quantityInput.value = '2';
  nameError.textContent = "";
  priceError.textContent = "";
  qtyError.textContent = "";
  productNameInput.classList.remove('invalid');
  unitPriceInput.classList.remove('invalid');
  quantityInput.classList.remove('invalid');
  calculateTotal();
});
// Run initial calculation to populate default summary
calculateTotal();
});

```

7. CODE EXPLANATION

Code	Explanation
<input type="number">	HTML5 input field configured with min, max, and step attributes for numeric entry.
document.addEventListener('DOMContentLoaded', ...)	Ensures JavaScript only executes after the entire HTML document object model has been fully parsed into memory.
addEventListener('input', ...)	Event listener that triggers instantly upon any user keystroke, character change, or input deletion.
parseFloat(value)	Converts a string representation of a number into a 64-bit IEEE 754 floating-point number.
parseInt(value, 10)	Parses string value to base-10 integer representation for discrete quantities.
isNaN(value)	Built-in JavaScript function checking whether a parsed value is

	mathematically invalid (Not-a-Number).
element.textContent	Safely retrieves or sets the raw textual content of an element without HTML parsing overhead or XSS vulnerabilities.
classList.add('invalid')	Programmatically appends a CSS error class to highlight offending input fields with red borders.
classList.remove('invalid')	Removes the CSS error class once valid data has been entered.
toLocaleString('en-IN')	Converts raw numeric float values into the standard Indian Rupee currency format with two fractional decimals.
dataset.name, dataset.price	Reads custom data-* attributes from preset HTML buttons to populate form fields dynamically.
onsubmit="return false;"	Prevents standard browser form submission that would cause an unwanted page reload.

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after executing the webpage in a browser.

Test Case	TC-01
Test / Input	Initial Page Load with Default Form Values (Wireless Gaming Mouse, Price: 1499.00, Qty: 2)
Expected Result	The invoice summary displays Subtotal: ₹2,998.00, GST (18%): ₹539.64, and Grand Total: ₹3,537.64 dynamically.
Actual Result	Page loaded and automatically rendered all summary fields and active pulse indicator without errors.
Status	Pass

TC-01 Output:

Interactive Product Calculator

Product Details

Product Name

Unit Price (₹)

Quantity

Quick Presets:

Live Invoice Summary

Product Name: **Wireless Gaming Mouse**

Unit Price: **₹1,499.00**

Quantity: **2**

Subtotal: **₹2,998.00**

GST (18%): **₹539.64**

Grand Total: **₹3,537.64**

Test Case	TC-02
Test / Input	Dynamic Quantity Change: Increase Quantity from 2 to 5 via keyboard input
Expected Result	Subtotal recalculates immediately to ₹7,495.00, GST to ₹1,349.10, and Total to ₹8,844.10 without page reload.
Actual Result	The total amount updated instantly on keystroke with zero screen refresh or latency.
Status	Pass

TC-02 Output:

Interactive Product Calculator

Product Details

Product Name

Unit Price (₹)

Quantity

Quick Presets:

Live Invoice Summary

Product Name: **Wireless Gaming Mouse**
Unit Price: ₹1,499.00
Quantity: 2
Subtotal: ₹2,998.00
GST (18%): ₹539.64
Grand Total: **₹3,537.64**

Interactive Product Calculator

Product Details

Product Name

Unit Price (₹)

Quantity

Quick Presets:

Live Invoice Summary

Product Name: **Wireless Gaming Mouse**
Unit Price: ₹1,499.00
Quantity: 5
Subtotal: ₹7,495.00
GST (18%): ₹1,349.10
Grand Total: **₹8,844.10**

Test Case	TC-03
Test / Input	Negative / Zero Input Validation: Enter Quantity = 0 or -2
Expected Result	Red border appears around quantity input, qtyError displays "Quantity must be at least 1.", Total resets to ₹0.00.
Actual Result	Input highlighted red, descriptive error string rendered, calculations safely guarded.
Status	Pass

TC-03 Output:

Interactive Product Calculator

Product Details

Product Name
Wireless Gaming Mouse

Unit Price (₹)
1499.00

Quantity
0

Quantity must be at least 1.

Quick Presets:

Keyboard
Mouse
USB Hub

Reset Form

Live Invoice Summary

Product Name:	Wireless Gaming Mouse
Unit Price:	₹1,499.00
Quantity:	2
Subtotal:	₹2,998.00
GST (18%):	₹539.64
Grand Total:	₹0.00

9. OUTPUT

Output 1: Interactive Product Calculator Webpage Output

Interactive Product Calculator

Product Details

Product Name
Mechanical Keyboard

Unit Price (₹)
3499.00

Quantity
2

Quick Presets:

Keyboard
Mouse
USB Hub

Reset Form

Live Invoice Summary

Product Name:	Mechanical Keyboard
Unit Price:	₹3,499.00
Quantity:	2
Subtotal:	₹6,998.00
GST (18%):	₹1,259.64
Grand Total:	₹8,257.64

Figure 1: Full Webpage Output of Interactive Product Calculator

10. RESULT

Thus, the **Interactive Product Calculator** webpage was successfully developed, styled, and verified using HTML5, CSS3, and JavaScript. Event listeners (input and change) and dynamic DOM updates were implemented to calculate order totals in real time without refreshing the webpage.